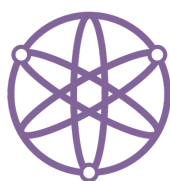




**POLITECHNIKA
RZESZOWSKA**
im. IGNACEGO ŁUKASIEWICZA



WYDZIAŁ
**MATEMATYKI
I FIZYKI STOSOWANEJ**
POLITECHNIKI RZESZOWSKIEJ

Zadanie Programistyczne nr 11

*Wyznaczanie długości najkrótszego podciągu
o sumie większej niż k*

Autor:

Alan Gancarczyk

Grupa: P03

Rzeszów, 29 stycznia 2026

1 Treść Zadania

Dla zadanego ciągu liczb naturalnych, znajdź długość najkrótszego ciągu, którego suma jest większa niż zadana liczba.

Przykład wejściowy: 1,2,3,4,5,6,7,8 k=20

Wyjście: 6, 7, 8

2 Analiza Problemu

W pierwszym podejściu postąpimy metodą „siłową” i zaimplementujemy rozwiązanie, które „narzuca się” samo. Utworzymy zmienną pomocniczą `min_len`, w której przechowywana będzie aktualna najmniejsza długość znalezionej podciągu. Następnie, metodycznie, sprawdzając każdy możliwy fragment tablicy za pomocą dwóch pętli, obliczymy sumę jego elementów. Jeśli suma ta okaże się większa od zadanej wartości `k`, to obliczymy długość tego podciągu i porównamy ją z wartością przechowywaną w zmiennej `min_len`. Jeżeli nowa długość jest mniejsza, przypiszemy ją do tej zmiennej, zapamiętując jednocześnie indeksy początku i końca w zmiennych `k_i` oraz `k_j`.

Należy zwrócić uwagę na problem inicjalizacji zmiennej `min` i odpowiednią wartością. Jaką wartość przypisać tej zmiennej przed analizą ciągu? Ponieważ poszukujemy minimum, zainicjowanie zmiennej zerem byłoby błędem, gdyż żaden wynik nie byłby od niej mniejszy. Zmienną `min_len` inicjalizujemy więc wartością $n + 1$, ponieważ jest to wartość większa od długości całej tablicy, a więc niemożliwa do osiągnięcia przez jakikolwiek rzeczywisty podciąg.

Należy również rozważyć przypadek „nieszczęśliwych” danych, gdzie żaden podciąg nie spełnia warunku sumy. Na ten wypadek zmienne indeksów `k_i` i `k_j` inicjalizujemy wartością -1. Jeśli po zakończeniu pętli wartości te nadal będą wynosić -1, będzie to informacja dla algorytmu, że nie znaleziono podtablicy spełniającej kryteria zadania.

Dodatkowo algorytm powinien uwzględniać optymalizację: w momencie, gdy suma elementów przekroczy wartość `k`, przerywamy pętlę wewnętrzną instrukcją `break`. Jest to uzasadnione tym, że dalsze dodawanie liczb naturalnych tylko zwiększyłoby długość ciągu, podczas gdy naszym celem jest znalezienie ciągu najkrótszego.

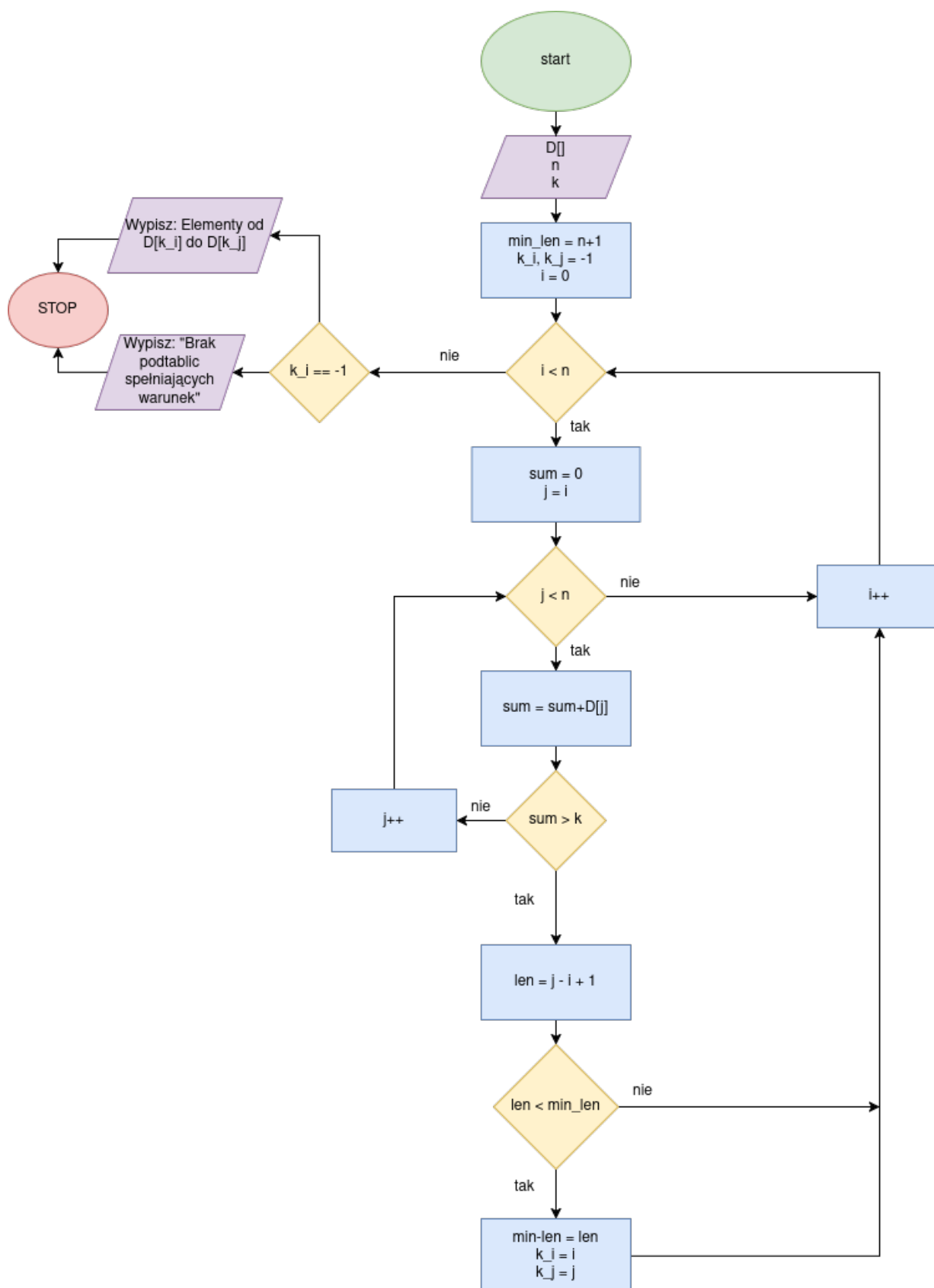
Dane wejściowe:

- `D[]` - Tablica wypełniona ciągiem liczb naturalnych
- `n` - rozmiar tablicy
- `k` - klucz do, którego podciągi mają być przyrównywane

Dane wyjściowe:

- Najkrótszy podciąg, którego suma jest większa niż zadana liczba `k`

3 Schemat Blokowy



4 Pseudokod

```

1  PROCEDURA solve(D, n, k)
2  min_len := n + 1
3  k_i := -1
4  k_j := -1
5
6  DLA i := 0 DO n - 1 WYKONUJ:
7      sum := 0
8      DLA j := i DO n - 1 WYKONUJ:
9          sum := sum + D[j]
10
11         JEŻELI sum > k TO:
12             len := j - i + 1
13
14             JEŻELI len < min_len TO:
15                 min_len := len
16                 k_i := i
17                 k_j := j
18             KONIEC JEŻELI
19
20         PRZERWIJ PĘTLĘ
21     KONIEC JEŻELI
22     KONIEC DLA
23     KONIEC DLA
24
25     WYPISZ "Wejście: "
26     DLA i := 0 DO n - 1 WYKONUJ:
27         WYPISZ D[i], " "
28     KONIEC DLA
29     WYPISZ " | k = ", k
30
31     JEŻELI k_i == -1 TO:
32         WYPISZ "Wyjście: Brak Podtablic spełniających warunek"
33     W PRZECIWNYM RAZIE:
34         WYPISZ "Wyjście: "
35         DLA i := k_i DO k_j WYKONUJ:
36             WYPISZ D[i], " "
37         KONIEC DLA
38     KONIEC JEŻELI
39
40     WYPISZ "-----"
41 KONIEC PROCEDURY

```

Kod 1: Pseudokod

5 Ołówkowe Sprawdzenie Poprawności Algorytmu

<i>i</i>	<i>j</i>	<i>Elementy w sumie</i>	<i>Suma końcowa</i>	<i>suma >20</i>	<i>len</i>	<i>zmiana min len</i>
0	0-5	1+2+3+4+5+6	21	TAK	6	6
1	1-6	2+3+4+5+6+7	27	TAK	6	brak zmiany 6!>6
2	2-6	3+4+5+6+7	25	TAK	5	5
3	3-6	4+5+6+7	22	TAK	4	4
4	4-7	5+6+7+8	26	TAK	4	brak zmiany 4!>4
5	5-7	6+7+8	21	TAK	3	3
6	6-7	7+8	15	NIE	-	brak zmiany
7	7	8	8	NIE	-	brak zmiany

6 Wykazanie Poprawności Algorytmu

6.1 Wykazanie własności stopu (Zastosowanie półniezmiennika)

Aby udowodnić, że program zawsze kończy działanie, wskazujemy funkcję, która w każdym kroku pętli maleje i jest ograniczona od dołu.

6.1.1 Pętla zewnętrzna

Rozważmy pętlę iterującą zmienną i od 0 do $n - 1$. Jako funkcję ograniczenia (półniezmiennik) przyjmujemy:

$$V_1(i) = n - i$$

Zauważmy, że:

- $V_1(i)$ przyjmuje wartości naturalne (dla $i \leq n$).
- W każdej iteracji zmienna i jest inkrementowana, zatem $V_1(i)$ maleje dokładnie o 1.
- Gdy $i = n$, warunek pętli $i < n$ przestaje być spełniony.

Wniosek: Pętla zewnętrzna wykonuje się skończoną liczbę razy.

6.1.2 Pętla wewnętrzna

Rozważmy pętlę iterującą zmienną j od i do $n - 1$. Jako funkcję ograniczenia przyjmujemy:

$$V_2(j) = n - j$$

Zauważmy, że:

- $V_2(j)$ przyjmuje wartości naturalne.
- W każdej iteracji zmienna j rośnie, więc wartość funkcji $V_2(j)$ maleje.
- Dodatkowo istnieje instrukcja **break**, która wymusza natychmiastowe zakończenie pętli (co jest równoważne skokowi do stanu końcowego pętli).

Wniosek: Pętla wewnętrzna wykonuje się skończoną liczbę razy. **Podsumowanie:** Ponieważ obie pętle są skończone, cały algorytm posiada **własność stopu**.

6.2 Dowód poprawności częściowej (Zastosowanie niezmiennika)

Poprawność algorytmu wynika z faktu, że sprawdza on wszystkie możliwe punkty startowe podtablic, a dla każdego punktu startowego znajduje najkrótszą możliwą podtablicę spełniającą warunek.

6.2.1 Niezmiennik pętli zewnętrznej

Sformułujmy niezmiennik dla pętli sterowanej zmienną i :

Niezmiennik: Przed rozpoczęciem i -tej iteracji zmienna `min_len` przechowuje długość najkrótszej podtablicy o sumie $> k$, która rozpoczyna się na indeksie $x < i$. Jeśli żadna taka podtablica nie została znaleziona w zakresie indeksów startowych $0 \dots i - 1$, zmienna `min_len` wynosi $n + 1$.

- **Inicjalizacja:** Przed pierwszą iteracją ($i = 0$), zakres sprawdzonych podtablic jest pusty. Zmienna `min_len` jest ustawione na $n + 1$ (wartość większa niż jakakolwiek możliwa długość podtablicy), co jest zgodne z prawdą.
- **Utrzymanie:** W iteracji i pętla wewnętrzna szuka najkrótszego ciągu zaczynającego się dokładnie w i . Jeśli znajdzie ciąg o długości $len < min_len$, aktualizuje `min_len`. Po zakończeniu iteracji i , `min_len` przechowuje minimum globalne dla wszystkich początków z zakresu $0 \dots i$.
- **Zakończenie:** Gdy $i = n$, pętla kończy działanie. Zgodnie z niezmiennikiem, `min_len` zawiera minimalną długość podtablicy znalezionej dla wszystkich możliwych początków od 0 do $n - 1$.

6.2.2 Poprawność instrukcji break

Kluczowym elementem algorytmu jest instrukcja `break`. Należy wykazać, że nie pomija ona potencjalnych rozwiązań.

Dla ustalonego i , pętla wewnętrzna zwiększa j , akumulując sumę. Długość badanego ciągu wynosi $len = j - i + 1$. W momencie, gdy po raz pierwszy spełniony zostanie warunek $sum > k$, znaleziona zostaje podtablica zaczynająca się w i o najmniejszej możliwej długości. Każde dalsze zwiększanie j dałoby podtablicę o długości większej niż ta znaleziona. Ponieważ szukamy minimum globalnego, przerwanie pętli w tym momencie jest poprawne i nie powoduje utraty optymalnego rozwiązania.

6.3 Podsumowanie

Poprzez wykazanie warunku stopu oraz poprawności częściowej udowodniono, że algorytm jest poprawny.

1. Algorytm zawsze kończy działanie.
2. Algorytm przegląda każdy możliwy indeks startowy i , znajdując dla niego lokalne optimum, co gwarantuje znalezienie optimum globalnego.

7 Teoretyczne oszacowanie złożoności obliczeniowej

Analizując algorytm można zauważyć, że jego podstawowymi operacjami są dodawanie elementu do sumy oraz sprawdzenie warunku $sum > k$ w pętli wewnętrznej. Rozważmy przypadek pesymistyczny, w którym warunek przerywania pętli (`break`) nie zostaje spełniony wcześniej (tzn. podciągi są długie lub suma nie przekracza k). Łatwo policzyć, ile iteracji pętli wewnętrznej zostanie wykonanych:

- dla $i = 0$ (pętla od $j = 0$ do $n - 1$) będzie to n iteracji
- dla $i = 1$ (pętla od $j = 1$ do $n - 1$) będzie to $n - 1$ iteracji
- dla $i = 2$ (pętla od $j = 2$ do $n - 1$) będzie to $n - 2$ iteracji
- ...
- dla $i = n - 1$ (pętla od $j = n - 1$ do $n - 1$) będzie to 1 iteracja

A więc całkowita liczba operacji w najgorszym przypadku to suma $n + (n - 1) + (n - 2) + \dots + 1$.

Przypominając sobie zależność na sumę kolejnych liczb naturalnych:

$$\sum_{k=1}^n k = \frac{n(n+1)}{2}$$

możemy policzyć, że dla ciągu o n elementach algorytm będzie musiał wykonać:

$$\sum_{k=1}^n k = \frac{n^2 + n}{2} = \frac{1}{2}n^2 + \frac{1}{2}n$$

Otrzymujemy w ten sposób złożoność obliczeniową rzędu $O(n^2)$.

8 Implementacja Algorytmu W Języku C++

8.1 Prosta Implementacja W Języku C++

```
1 #include <iostream>
2 using namespace std;
3
4 void solve(int D[], int n, int k) {
5     int min_len = n + 1;
6     int k_i = -1;
7     int k_j = -1;
8     int steps=0;
9     for (int i = 0; i < n; i++) {
10         int sum = 0;
11         for (int j = i; j < n; j++) {
12             sum += D[j];
13
14             if (sum > k) {
15                 int len = j - i + 1;
16
17                 if (len < min_len) {
18                     min_len = len;
19                     k_i = i;
20                     k_j = j;
21                 }
22                 break;
23             }
24         }
25         cout << "Wejście: ";
26         for (int i = 0; i < n; i++) {
27             cout << D[i] << " ";
28         }
29         cout << " | k = " << k << endl;
30
31         if (k_i == -1) {
32             cout << "Wyjście: Brak Podtablic spełniających warunek" << endl;
33         } else {
34             cout << "Wyjście: ";
35             for (int i = k_i; i <= k_j; i++) {
36                 cout << D[i] << " ";
37             }
38             cout << endl;
39         }
40
41         cout << "-----" << endl;
42     }
43 }
```

Kod 2: Prosta Implementacja

8.2 Eksperymentalne Sprawdzenie Wydajności Algorytmu

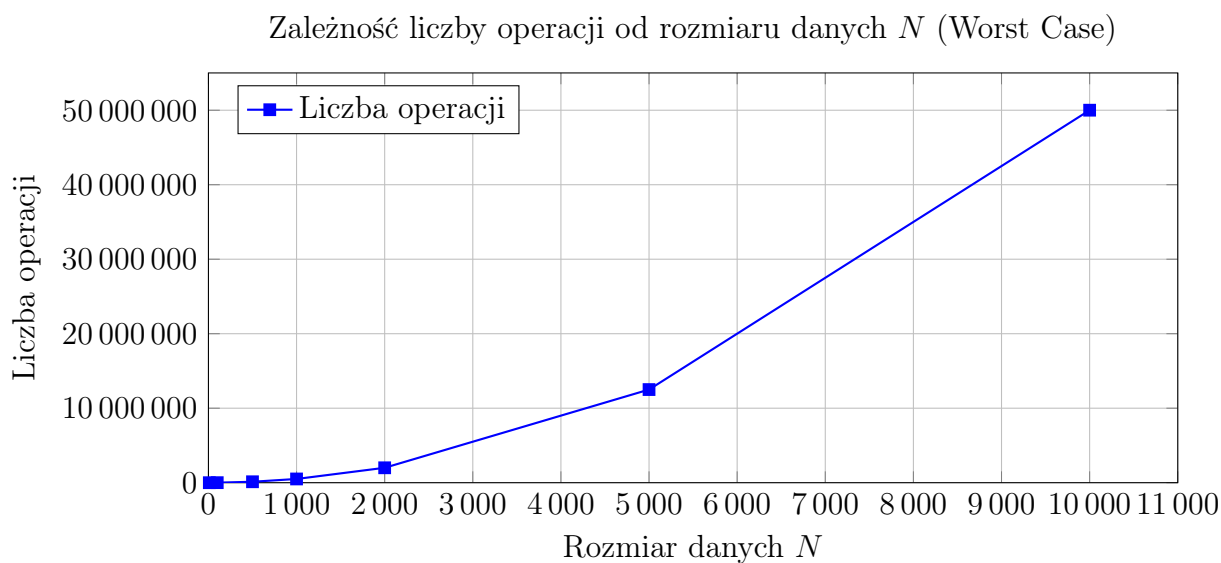
```
1 #include <iostream>
2 #include <vector>
3 #include <chrono>
4
5 using namespace std;
6 using namespace std::chrono;
7
8 void solve(const vector<int>& D, int n, int k) {
9     int min_len = n + 1;
10    long long steps = 0; // Licznik operacji
11
12    //start "zegarka"
13    auto start = high_resolution_clock::now();
14
15    for (int i = 0; i < n; i++) {
16        int sum = 0;
17        for (int j = i; j < n; j++) {
18            steps++; // Zliczamy operację główną
19
20            sum += D[j];
21            if (sum > k) {
22                int len = j - i + 1;
23                if (len < min_len) {
24                    min_len = len;
25                }
26                break;
27            }
28        }
29    }
30
31    auto stop = high_resolution_clock::now();
32    auto duration = duration_cast<nanoseconds>(stop - start);
33    cout << "N = " << n;
34    cout << " | Operacje: " << steps;
35    cout << " | Czas: " << double(duration.count())/1000000 << " ms" <<
36    endl;
37 }
38
39 int main() {
40     int rozmiary[] = {10, 100, 500, 1000, 2000, 5000, 10000};
41     int k = INT32_MAX;
42
43     cout << "test Wydajności" << endl;
44
45     for (int n : rozmiary) {
46         vector<int> D(n);
47         for(int i=0; i<n; i++) D[i] = rand() % 100;
48         solve(D, n, k);
49     }
50     return 0;
51 }
```

Kod 3: Sprawdzenie wydajności

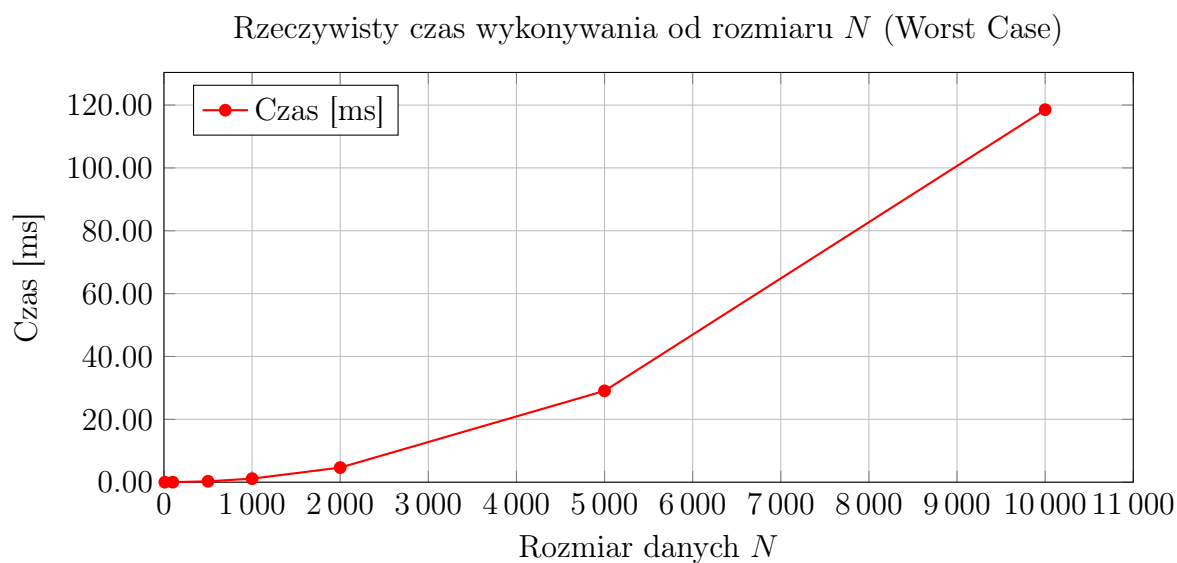
Sprawdzamy wydajność dla przypadku pesymistycznego. Ustawiamy zmienną k na wartość niemożliwie dużą przez co pętla wewnętrzna jest zmuszona do sprawdzenia wszyst-

kich elementów do końca.

8.2.1 Wykresy Wydajności Dla Danych Pesymistycznych



Rysunek 1: Liczba operacji (liczba wykonań pętli wewnętrznej)



Rysunek 2: Czas wykonywania algorytmu

Jak widać na powyższych wykresach, liczba obliczeń i czas wykonywania algorytmu rośnie kwadratowo co dodatkowo potwierdza, że złożoność obliczeniowa z jaką mamy tutaj do czynienia jest rzędu $O(n^2)$